Prompt Engineering
Fondamenti

Boolean Master · Lezione 11

Che cos’è il prompt engineering

Scrivere bene le istruzioni per il modello. È un'abilità, con regole pratiche.

LEZIONE 10

Come funziona

Il modello completa il contesto che gli dai, un token alla volta, dentro una finestra.

→

LEZIONE 11 · OGGI

Come si usa

Quel contesto lo scrivi tu. Scriverlo bene, in modo ripetibile, è il prompt engineering.

Vuol dire scrivere le istruzioni in modo chiaro, testabile e riusabile. Non è cercare la frase magica: è un metodo.

Cosa vediamo oggi

Niente codice oggi: il codice è sabato. Oggi i fondamenti del prompting.

01
Anatomia
I tre ruoli e il contesto.

02
CIAO
Una struttura per scrivere bene.

03
Le tecniche
Role, few-shot, negative.

04
Temperature
Controllare la generazione.

05 · LIVE
Demo
Il playground dal vivo.

06
Iterare
E gli errori da evitare.



1. Anatomia di una richiesta

Cosa riceve davvero il modello

I tre ruoli

Ogni richiesta a un modello è fatta di tre parti.

SYSTEM

Le istruzioni di base

Date prima della conversazione: ruolo del modello, regole, formato della risposta.

USER

Il messaggio dell'utente

La domanda o il compito che chiedi al modello.

ASSISTANT

La risposta del modello

Quello che il modello genera in risposta.

Una conversazione

I tre ruoli che si alternano, sempre sotto lo stesso system.

`SYSTEM` Sei un assistente che classifica i ticket di supporto. Rispondi conciso.

`USER` Il login non funziona per il 30% degli utenti.

`ASSISTANT` CRITICO, impatto alto, escalation immediata.

`USER` E un refuso nella pagina About?

La conversazione cresce, ma a ogni nuovo messaggio il modello riceve **tutto da capo**: il system più tutti i turni precedenti.

↑ tutto questo entra in input, ogni volta



Il contesto lo gestiamo noi

Il modello non ha memoria: ogni chiamata riparte da zero.

SOLO L'ULTIMO MESSAGGIO

USER E i campi opzionali?

ASSISTANT Di quali campi parli?

Il modello non sa di cosa stai parlando.

CON TUTTA LA CRONOLOGIA

USER Come valido uno schema in Python?

ASSISTANT Con Pydantic: definisci un BaseModel coi campi tipizzati.

USER E i campi opzionali?

ASSISTANT Usali con Optional e default None.

Le chat (ChatGPT, Claude.ai) rimandano la cronologia al posto tuo. Sabato, scrivendo codice, la cronologia la costruiamo noi a ogni chiamata.

IN BREVE Il modello non ricorda niente tra una chiamata e l'altra: la cronologia gliela passiamo noi, ed è un lavoro che ci riguarda soprattutto nel codice.

Chi scrive il system prompt?

La differenza tra usare un tool pronto e scrivere la tua app.

TOOL PRONTO · ChatGPT, Claude.ai

SYSTEM [bloccato] scritto dal provider

Non lo vedi, non lo controlli. Lavori sopra il system di qualcun altro.

API · la tua app

SYSTEM **Io scrivi tu**

Controllo totale sul contesto che mandi al modello.

Per questo un esperimento fatto in chat non equivale a una chiamata via API: sopra c'è un system diverso.

Se la richiesta è ambigua

A volte il modello può chiederti chiarimenti, ma spesso non lo fa.

`"scrivimi una funzione che pulisce i dati"`

- in Python? rimuove i null?
- in JS? toglie i duplicati?
- normalizza le stringhe?

Spesso non chiede: sceglie l'interpretazione più probabile e va avanti. Non puoi contarci, quindi conviene essere chiari fin da subito.

chiedere senza esempi si chiama **zero-shot:** è il punto di partenza

Zero-shot e few-shot

Questi modelli sono generalisti: li indirizzi al momento della richiesta, non li riaddestri.

Un LLM non è addestrato per il tuo compito specifico: ha imparato a prevedere il prossimo token su enormi quantità di testo. Per fargli fare un compito glielo spieghi nel prompt, al momento della richiesta.

ZERO-SHOT

Chiedi direttamente, senza esempi.

```
Che sentiment ha questa frase?
"Spedizione lenta, prodotto ottimo." → ?
```

FEW-SHOT

Metti due o tre esempi: il modello coglie il pattern.

```
"Consegna puntuale!" → positivo
"Mai più." → negativo
"Lenta ma ottimo." → ?
```

IN BREVE Zero-shot e few-shot non riaddestrano il modello: lo guidano col prompt (in-context learning, i pesi non cambiano).

Mi raccomando!!!

Una parola che vi tradisce: **non** chiamatelo "training".

IL PROVIDER · OpenAI, Anthropic

Ha addestrato il modello

Training (e fine-tuning) modifica i **pesi** su enormi dataset. Costoso e lungo: lo fa chi costruisce il modello.

NOI

Lo istruiamo col prompt

Al momento della richiesta, in-context. **I pesi non si toccano**: gli stiamo solo dando istruzioni.

NON DIRE ~~"ho addestrato / fine-tunato un modello"~~

DI' "l'ho istruito via prompt"

Dire "ho addestrato un modello" quando hai solo scritto un prompt fa trasparire che non hai capito come funziona.



2. CIAO: la struttura di un buon prompt

Una checklist per non lasciare buchi

Il framework CIAO

Un aiuto per ricordare le quattro cose che un buon prompt contiene.

C

Contesto

Chi sei, su cosa stai lavorando, qual è la situazione.

I

Istruzione

Cosa vuoi che il modello faccia, in modo netto.

A

Aspettativa

Formato, tono, lunghezza, vincoli.

O

Output di esempio

Mostra un esempio di cosa ti aspetti.

Vago vs CIAO

Lo stesso compito, chiesto male e chiesto bene.

PROMPT VAGO

```
"scrivimi una funzione che pulisce i dati"
```

Il modello deve indovinare quale linguaggio, cosa vuol dire "pulire" e che formato restituire. Di solito esce qualcosa di generico, che poi devi rifare.

PROMPT CON CIAO

- **C** parser di log in Python 3.12
- **I** estrai timestamp e livello di errore da ogni riga
- **A** tipizzata, con docstring, zero dipendenze
- **O** `parse(line) → LogEntry | None`

Linguaggio, compito, vincoli e formato sono scritti nero su bianco. Il modello restituisce una funzione Python tipizzata e documentata, pronta da usare.

Placeholders

Scrivi il prompt una volta, riusalo su tanti dati diversi.

UN PROMPT, SCRITTO UNA VOLTA

```
<ruolo> classifica i ticket
<vincoli> gravità + azione
<ticket> {{ticket}}
```

Il placeholder è il buco che riempi con il dato di volta in volta.

"server giù da 2 ore"	CRITICO
"refuso pagina About"	BASSO
"export PDF lento"	MEDIO

stesso prompt · cambia solo il dato

È così che lo stesso prompt diventa un pezzo di un'automazione: una classificazione per ogni ticket in entrata.

3. Le tecniche

Indirizzare il modello verso la risposta giusta



Role prompting

Stesso modello, stessa domanda: il ruolo che assegni cambia la risposta.

`"Come miglioro questa query?"`

SENIOR DEV

"Sei un dev esperto di performance"

Parla di indici, EXPLAIN, piani di esecuzione.

REVIEWER SEVERO

"Agisci come un reviewer che cerca i bug"

Elenca rischi, edge case, cosa può rompersi.

SPIEGA SEMPLICE

"Spiega a un principiante"

Linguaggio accessibile, un passo alla volta.

In API il ruolo si mette nel system prompt. Serve a dare una direzione, invece di lasciare il modello sul suo tono neutro di default.

Few-shot: quando conviene

Gli esempi servono a comunicare i tuoi criteri, non a insegnare da zero.

ES. 1 "server giù da 2 ore" → **CRITICO**

ES. 2 "refuso pagina About" → **BASSO**

ORA "login giù per il 30%" → **CRITICO**

Conviene

Sui task con criteri tuoi (la classificazione fatta a modo tuo): gli esempi dicono al modello cosa intendi.

È sprecato

Sui task che il modello fa già bene (tradurre): gli esempi sono solo token in più.

Ogni esempio è contesto in più (più token, più costo): tienine pochi, due o tre, e rappresentativi.

IN BREVE Gli esempi aiutano quando comunicano criteri tuoi; sui compiti che il modello sa già fare sono solo token in più.

Istruzioni negative

Dire cosa NON fare è utile quanto dire cosa fare.

A volte la via più diretta è dire al modello cosa **non** deve fare. Funziona bene come le istruzioni positive, e spesso le due si combinano nello stesso prompt.

SENZA

`"Scrivi la funzione di parsing."`

"Certamente! Ecco una funzione che...", seguito da una spiegazione lunga.

CON ISTRUZIONE NEGATIVA

`"Scrivi la funzione di parsing. Solo codice, niente preamboli né spiegazioni."`

Esce direttamente il blocco di codice.

Spesso si combinano: una positiva ("solo codice") più una negativa ("niente preamboli"). Altre utili: "niente emoji", "non superare le 100 parole", "non inventare dati se non sei sicuro".

4. Controllare la generazione

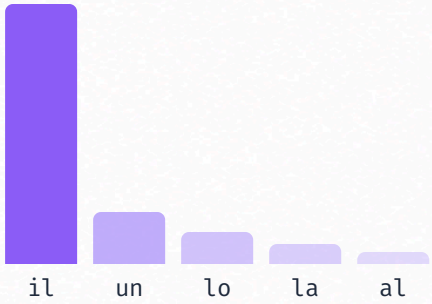
La temperature e gli altri parametri



La temperature

A ogni passo c'è una distribuzione di probabilità. La temperature decide quanto seguirla alla lettera.

Temperature bassa · 0.2



Conservativa, ripetibile.

Temperature alta · 1.0



Varia, creativa.

Quale temperature quando

Quasi sempre il default. Abbassala per risposte ripetibili, alzala per più varietà.



0 – 0.3 · RIPETIBILITÀ

Classificazioni, estrazione dati

Stesso input, stessa risposta. Per i ticket: temperature bassa.

DEFAULT

Va bene quasi sempre

Nella maggior parte dei casi non si tocca.

0.7 – 1.0 · CREATIVITÀ

Brainstorming, alternative

Quando vuoi varietà ed esplorazione.

Gli altri parametri

Esistono altre manopole, ma quasi sempre restano sul default.

`top-p`	Tiene solo i token più probabili fino a coprire una certa fetta di probabilità. Come la temperature, regola quanta varietà permetti.
`top-k`	Considera solo i k token più probabili a ogni passo, scartando la coda.
`max tokens`	Il tetto di lunghezza della risposta.
`stop`	Una sequenza di testo che, se compare, ferma la generazione.

Si regolano via **API o playground**, non nella chat consumer (lì la temperature è fissa). Sabato diventano parametri Python.

`IN BREVE` Nella pratica quasi sempre bastano i default: queste manopole servono solo in casi particolari.



Demo

DEMO DAL VIVO **aistudio.google.com** modello: Gemini Flash

1. **Struttura**: System = `squad <nome> · <avviso>`, niente preamboli, + esempi con le tue squad → output pulito (`squad Vega · …`)
2. **Few-shot (flip)**: togli gli esempi → inventa un nome; rimettili → torna `squad Vega`, che poteva sapere solo da lì
3. **Routing controintuitivo**: "export PDF lento" → `squad Lyra` (billing), non UI: presa dagli esempi
4. **Temperature**: `0` → riga identica a ogni rigenerazione; `2` → la frase di avviso cambia ogni volta → in produzione si va a 0



5. Iterare e gli errori

Il primo tentativo è un punto di partenza



Iterare sul risultato

Raramente il primo output va bene. Lo si sistema in due o tre passaggi.

Leggi cosa torna, vedi cosa manca, e chiedi una correzione precisa: cambia il tono, aggiungi un dato, taglia una parte. Esempio: una email che annuncia un ritardo a un cliente.

v1

Generica e troppo formale.

→ "più colloquiale, aggiungi la nuova scadenza"

v2

Tono giusto, nuova timeline.

→ "aggiungi cosa è già stato fatto"

v3

Elenco progressi, pronta da inviare.

Non aspettarti la risposta perfetta al primo colpo: la conversazione è lo strumento per arrivarci.

Cinque errori da evitare

I cinque modi più rapidi per ottenere un output deludente.

1	Prompt di una riga per task complessi	→ dai contesto
2	Non specificare il formato di output	→ specificalo
3	Dare per scontato che conosca il tuo contesto	→ fornisciglielo
4	Non verificare l'output: codice e numeri possono essere sbagliati	→ allucinazioni (L10)
5	Fermarsi al primo tentativo	→ itera

Riepilogo

Quattro cose per scrivere un buon prompt.

CAP. 2

Struttura

CIAO: una checklist.

→

CAP. 3

Tecniche

Role, few-shot, negative.

→

CAP. 4

Controllo

La temperature.

→

CAP. 5

Iterazione

Due o tre giri mirati.

Non serve una frase magica: serve scrivere il contesto in modo chiaro e poi sistemarlo.



Un buon prompt, tutto insieme

I pilastri del riepilogo in un unico prompt: CIAO, role, few-shot, istruzioni negative.

```
[CONTESTO + RUOLO] Sei il sistema di triage dei ticket di un'app SaaS. Aree note: auth, billing, ui, infra.

[ISTRUZIONE] Per ogni ticket assegna gravità, area e una micro-azione.

[FORMATO + VINCOLI] Rispondi SOLO con una riga: <P0|P1|P2|P3> · <area> · <azione, max 5 parole>. Niente preamboli, niente spiegazioni, niente emoji.

[ESEMPI]
"Pagamenti rifiutati per tutti" → P0 · billing · escalation on-call
"Refuso nel footer" → P3 · ui · backlog

[TICKET] {{ticket}}
```

STRUTTURA Le sezioni sono i capitoli di CIAO.

ROLE "Sei il sistema di triage…": dà una direzione.

FEW-SHOT Due esempi insegnano i tuoi criteri.

NEGATIVE "Niente preamboli…" tiene l'output pulito.

Intorno al prompt: temperature bassa perché è un classificatore (controllo), più 2-3 giri di rifinitura se serve (iterazione).

Prompting avanzato

Tecniche e problemi che vedrai più avanti nel corso, con lezioni dedicate nel Modulo 2.

CHAIN-OF-THOUGHT

Far ragionare il modello passo passo: meglio sui compiti complessi.

→ Prompt Avanzato

SELF-CONSISTENCY

Lo stesso prompt N volte, vince la risposta più frequente.

→ Prompt Avanzato

STRUCTURED OUTPUT

Non basta chiedere "in JSON": si vincola a livello di API.

→ Structured Output

CONTEXT ROT

Le conversazioni lunghe degradano: il modello perde il centro.

→ contesto pulito (L10)

Kahoot

Verifichiamo con un Kahoot: 5 domande sulla lezione.

IL DOCENTE APRE E OSPITA QUI

create.kahoot.it/details/6a3f49f5-f6be-43f6-9068-5b658280161a

Gli studenti entrano su **kahoot.it** con il PIN che compare a schermo.

Sabato: la prima applicazione in Python

Gli stessi concetti di oggi, scritti in codice.

OGGI · A PAROLE

Il playground

System e User nei pannelli, la temperature su un cursore. Si sperimenta a mano.

SABATO · LEZIONE 12

Prima applicazione AI

I ruoli diventano codice, la temperature un parametro. In più: gestione degli errori e streaming.

`openai` `Pydantic` `async`